Priority Queue의 데이터로 하는 Priority Queue의 동작 STL 코드

웹 페이지 10개 읽음

C++ STL의 `priority_queue`를 사용하여 **우선순위 큐 자체를 데이터로 저장**하는 코드는 흔하지 않은 활용법입니다. 아래에서 기본적인 사용법과 함께, `priority_queue`를 요소로 갖는 `vector`를 활용하는 예시 코드를 설명드리겠습니다.

📚 기본 개념 및 STL 코드 예시

C++ STL의 `priority_queue`는 기본적으로 **최대 힙(Max Heap)**으로 동작하며, 가장 큰 값이 가장 높은 우선순위를 가집니다 1 4 8. `priority_queue`를 다른 컨테이너의 요소로 사용하려면, 템플릿 인자를 중첩하여 선언합니다.

다음은 `priority_queue<int>`를 데이터로 하는 `vector`의 예시 코드입니다.

cpp

```cpp
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int main() {
    // priority_queue를 요소로 갖는 vector 선언
    vector<priority_queue<int>> vec_of_pq;

    // 첫 번째 priority_queue 생성 및 데이터 추가
    priority_queue<int> pq1;
    pq1.push(30);
    pq1.push(10);
    pq1.push(20);
    vec_of_pq.push_back(pq1); // vector에 추가

    // 두 번째 priority_queue 생성 및 데이터 추가
    priority_queue<int> pq2;
    pq2.push(5);
    pq2.push(15);
    vec_of_pq.push_back(pq2);

    // vector에 저장된 각 priority_queue의 내용 출력
    for (int i = 0; i < vec_of_pq.size(); ++i) {
```

```
        cout << "Priority Queue " << i + 1 << " top: " << vec_of_pq[i].top() << endl;

        // 해당 큐의 모든 요소를 출력하려면 (복사본으로 작업)
        priority_queue<int> temp = vec_of_pq[i];
        cout << "All elements: ";
        while (!temp.empty()) {
            cout << temp.top() << " ";
            temp.pop();
        }
        cout << endl;
    }

    return 0;
}
```

💡 주의사항 및 활용 팁

이 코드를 사용할 때는 몇 가지 점을 유의해야 합니다.

- **직접 접근의 제한**: `vec_of_pq[i]` 를 통해 특정 우선순위 큐에 직접 접근은 가능하지만, **STL 컨테이너를 값으로 저장**하므로 복사 비용이 발생할 수 있습니다.
- **복사본으로 순회**: 예시 코드에서와 같이 `temp` 복사본을 만들어 모든 요소를 출력합니다. 원본 `vec_of_pq[i]` 에서 `pop()` 을 직접 호출하면 데이터가 사라지므로 주의해야 합니다 4 8 .
- **실용성**: 여러 개의 독립적인 우선순위 큐를 관리해야 하는 시나리오(예: 여러 스레드별 작업 큐 관리)에서 이러한 구조를 활용할 수 있습니다.

🛠️ 다른 컨테이너와의 조합

`priority_queue` 는 `vector` 외에도 `deque` 와 같은 다른 컨테이너와도 조합할 수 있습니다 9 . 필요에 따라 `vector<priority_queue<int, vector<int>, greater<int>>>` 형태로 **최소 힙(Min Heap)**을 저장하는 벡터를 만드는 등 다양한 활용이 가능합니다 2 8 10 .

💎 요약

`priority_queue` 의 `priority_queue` 활용은 특정 관리 시나리오에 유용합니다. 선언 시 템플릿 중첩과 데이터 접근 시 복사본 사용에 유의하면 효과적으로 활용할 수 있습니다.

특정 응용 사항(예: 최소 힙 사용, 사용자 정의 비교 함수)에 대한 추가 설명이 필요하시면 말씀해 주세요.

웹페이지 10개

priority_queue <priority_queue<int>> DeepPQ ; 의 동작을 확인할 수 있는 C++ 예제 코드

`priority_queue <priority_queue<int>> DeepPQ` 와 같은 **중첩 우선순위 큐**의 동작을 확인할 수 있는 C++ 예제 코드입니다.

cpp

```cpp
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int main() {
    // 중첩 우선순위 큐 선언
    // priority_queue<priority_queue<int>> DeepPQ;  // 이렇게 사용하면 컴파일 에러 발생

    // 대신 다음과 같이 선언해야 합니다
    // priority_queue의 기본 컨테이너는 vector이며, less<>를 사용합니다
    priority_queue<priority_queue<int>, vector<priority_queue<int>>, less<priority_queue<int>>> DeepPQ;

    cout << "=== 중첩 우선순위 큐(DeepPQ) 동작 테스트 ===" << endl;

    // 여러 개의 priority_queue 생성
    priority_queue<int> pq1;
    pq1.push(30);
    pq1.push(10);
    pq1.push(20);

    priority_queue<int> pq2;
    pq2.push(5);
    pq2.push(15);
    pq2.push(25);
    pq2.push(35);

    priority_queue<int> pq3;
    pq3.push(100);
    pq3.push(50);

    // 중첩 큐에 각 큐 추가
    cout << "\n1. 중첩 큐에 3개의 priority_queue 추가:" << endl;
    DeepPQ.push(pq1);
```

```
    DeepPQ.push(pq2);
    DeepPQ.push(pq3);

    cout << "현재 중첩 큐 크기: " << DeepPQ.size() << endl;

    // 중첩 큐의 top 요소 확인 및 출력
    cout << "\n2. 중첩 큐의 top 요소 확인:" << endl;
    if (!DeepPQ.empty()) {
        priority_queue<int> topPQ = DeepPQ.top();
        cout << "가장 높은 우선순위를 가진 큐의 top 값: " << topPQ.top() << endl;

        // 해당 큐의 모든 요소 출력
        cout << "해당 큐의 모든 요소: ";
        priority_queue<int> temp = topPQ;
        while (!temp.empty()) {
            cout << temp.top() << " ";
            temp.pop();
        }
        cout << endl;
    }

    // 중첩 큐의 모든 요소 순회 (실제로는 pop 해야만 접근 가능)
    cout << "\n3. 중첩 큐의 모든 요소 순회:" << endl;
    int count = 1;

    // 중첩 큐를 복사해서 작업 (원본 유지)
    auto tempDeepPQ = DeepPQ;

    while (!tempDeepPQ.empty()) {
        priority_queue<int> currentPQ = tempDeepPQ.top();
        tempDeepPQ.pop();

        cout << "큐 " << count++ << " - ";
        cout << "원소 개수: " << currentPQ.size() << ", ";
        cout << "최대값: " << currentPQ.top() << ", ";

        // 현재 큐의 모든 요소 출력
        cout << "모든 요소: ";
        priority_queue<int> temp = currentPQ;
        while (!temp.empty()) {
            cout << temp.top() << " ";
            temp.pop();
        }
        cout << endl;
    }

    // 원본 중첩 큐에서 요소 제거
```

```
    cout << "\n4. 원본 중첩 큐에서 pop 작업:" << endl;
    while (!DeepPQ.empty()) {
        priority_queue<int> topPQ = DeepPQ.top();
        cout << "제거되는 큐의 top 값: " << topPQ.top() << endl;
        DeepPQ.pop();
        cout << "pop 후 중첩 큐 크기: " << DeepPQ.size() << endl;
    }

    return 0;
}
```

🔍 컴파일 에러 해결 방법

위 코드에서 주석 처리된 `priority_queue<priority_queue<int>> DeepPQ;` 는 **컴파일 에러**가 발생합니다. 그 이유는:

```
cpp

// 컴파일 에러 발생 - 비교 연산자가 정의되지 않음
// priority_queue<priority_queue<int>> DeepPQ;

// 올바른 선언 방법
priority_queue<priority_queue<int>, vector<priority_queue<int>>, less<priority_queue<
int>>> DeepPQ;
```

🎯 중첩 우선순위 큐의 비교 기준

중첩 우선순위 큐는 **내부 큐들 간의 비교**를 어떻게 할까요?

```
cpp

#include <iostream>
#include <queue>
#include <vector>

using namespace std;

// priority_queue 비교를 위한 사용자 정의 비교 함수
struct PriorityQueueComparator {
    bool operator()(const priority_queue<int>& a, const priority_queue<int>& b) const
{
        // 각 큐의 top 값을 기준으로 비교
        if (a.empty() && b.empty()) return false;
        if (a.empty()) return true;    // 빈 큐가 더 작음
```

```
        if (b.empty()) return false;   // 빈 큐가 더 작음

        return a.top() < b.top();  // 최대값 기준으로 비교
    }
};

int main() {
    // 사용자 정의 비교 함수를 사용한 중첩 큐
    priority_queue<priority_queue<int>,
                   vector<priority_queue<int>>,
                   PriorityQueueComparator> CustomDeepPQ;

    cout << "=== 사용자 정의 비교 함수를 사용한 중첩 큐 ===" << endl;

    priority_queue<int> q1, q2, q3;

    q1.push(100);
    q1.push(50);

    q2.push(200);
    q2.push(150);

    q3.push(75);
    q3.push(125);

    CustomDeepPQ.push(q1);
    CustomDeepPQ.push(q2);
    CustomDeepPQ.push(q3);

    cout << "중첩 큐의 top에 있는 큐의 최대값: " << CustomDeepPQ.top().top() << endl;

    return 0;
}
```

💡 실행 결과 예시

```
text

=== 중첩 우선순위 큐(DeepPQ) 동작 테스트 ===

1. 중첩 큐에 3개의 priority_queue 추가:
현재 중첩 큐 크기: 3

2. 중첩 큐의 top 요소 확인:
가장 높은 우선순위를 가진 큐의 top 값: 100
해당 큐의 모든 요소: 100 50
```

```
3. 중첩 큐의 모든 요소 순회:
큐 1 - 원소 개수: 2, 최대값: 100, 모든 요소: 100 50
큐 2 - 원소 개수: 4, 최대값: 35, 모든 요소: 35 25 15 5
큐 3 - 원소 개수: 2, 최대값: 30, 모든 요소: 30 20 10

4. 원본 중첩 큐에서 pop 작업:
제거되는 큐의 top 값: 100
pop 후 중첩 큐 크기: 2
제거되는 큐의 top 값: 35
pop 후 중첩 큐 크기: 1
제거되는 큐의 top 값: 30
pop 후 중첩 큐 크기: 0
```

🎯 핵심 요점

1. **직접 선언 불가**: `priority_queue<priority_queue<int>>` 는 컴파일 에러 발생
2. **명시적 선언 필요**: 컨테이너와 비교 함수를 명시적으로 지정해야 함
3. **비교 기준**: 기본적으로 `priority_queue` 간의 비교 연산자가 필요
4. **실용성**: 실제 프로젝트에서는 이런 복잡한 구조보다 더 간단한 설계를 권장

이 코드를 통해 중첩 우선순위 큐의 동작 원리를 이해하실 수 있을 것입니다!

> priority_queue<priority_queue<int>> DeepPQ; 이렇게 사용하면 왜 컴파일 에러가 발생하는가?

`priority_queue<priority_queue<int>> DeepPQ;` 가 **컴파일 에러**가 발생하는 이유는 **템플릿 인자 추론**과 **비교 연산자** 문제 때문입니다.

🔍 컴파일 에러 발생 이유

1. priority_queue의 기본 템플릿 구조

```cpp
template<
    class T,
    class Container = std::vector<T>,
    class Compare = std::less<typename Container::value_type>
> class priority_queue;
```

기본 `priority_queue` 는 **3개의 템플릿 매개변수**를 갖습니다. 우리가 `priority_queue<int>` 라고 작성하면:

```cpp
priority_queue<int> // 실제로는:
priority_queue<int, vector<int>, less<int>>
```

2. 중첩 사용 시 문제점

```cpp
priority_queue<priority_queue<int>> DeepPQ;
// 컴파일러는 이를 다음과 같이 해석하려고 시도:
priority_queue<priority_queue<int>, vector<priority_queue<int>>, less<priority_queue<int>>>
```

문제: `less<priority_queue<int>>` 가 **컴파일되지 않습니다!**

🧪 실제 컴파일 에러 메시지 분석

```cpp
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<priority_queue<int>> DeepPQ;  // 컴파일 에러!
    return 0;
}
```

에러 메시지 예시:

```text
error: no match for 'operator<' (operand types are 'const std::priority_queue<int>' and 'const std::priority_queue<int>')
```

🔬 문제의 근본 원인

비교 연산자 부재

`priority_queue<int>` 타입에 대해 `operator<` **가 정의되어 있지 않습니다.**

```cpp
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq1, pq2;

    // 다음 비교 연산들이 모두 컴파일 에러!
    bool b1 = (pq1 < pq2);    // 에러!
    bool b2 = (pq1 > pq2);    // 에러!
    bool b3 = (pq1 == pq2);   // 에러!

    return 0;
}
```

💡 해결 방안 비교

❌ 에러 발생하는 코드

```cpp
priority_queue<priority_queue<int>> DeepPQ;  // 컴파일 에러!
```

✅ 해결 방법 1: 사용자 정의 비교 함수 제공

```cpp
#include <queue>
#include <vector>
using namespace std;

// priority_queue 비교를 위한 함수 객체
struct PQComparator {
    bool operator()(const priority_queue<int>& a,
                    const priority_queue<int>& b) const {
        // 각 큐의 크기로 비교하는 예시
        return a.size() < b.size();
    }
```

```
};

// 사용자 정의 비교 함수 사용
priority_queue<priority_queue<int>, vector<priority_queue<int>>, PQComparator> DeepP
Q;
```

✅ 해결 방법 2: 포인터 사용

cpp

```
#include <queue>
#include <vector>
#include <memory>
using namespace std;

// 포인터는 비교 가능하므로 컴파일 에러 없음
priority_queue<priority_queue<int>*> PtrDeepPQ;

// 또는 스마트 포인터
priority_queue<shared_ptr<priority_queue<int>>> SmartDeepPQ;
```

✅ 해결 방법 3: 래퍼 클래스 사용

cpp

```
#include <queue>
#include <vector>
using namespace std;

// 비교 연산자를 갖는 래퍼 클래스
struct ComparablePriorityQueue {
    priority_queue<int> pq;

    // 비교 연산자 정의
    bool operator<(const ComparablePriorityQueue& other) const {
        return !pq.empty() && !other.pq.empty() && pq.top() < other.pq.top();
    }
};

// 이제 컴파일 가능
priority_queue<ComparablePriorityQueue> DeepPQ;
```

🎯 실제 동작하는 예제

cpp

```cpp
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 크기 기준으로 priority_queue 비교
struct SizeComparator {
    bool operator()(const priority_queue<int>& a,
                    const priority_queue<int>& b) const {
        return a.size() < b.size();  // 크기가 큰 것이 높은 우선순위
    }
};

int main() {
    // 사용자 정의 비교 함수를 사용한 중첩 큐
    priority_queue<priority_queue<int>,
                   vector<priority_queue<int>>,
                   SizeComparator> DeepPQ;

    // 여러 큐 생성 및 추가
    priority_queue<int> pq1, pq2, pq3;

    pq1.push(1); pq1.push(2); pq1.push(3);  // 크기: 3
    pq2.push(
```